

Fireflies Clone — Revised Implementation Plan

Scaler SDE Fullstack assignment. This version fixes the schema, API, scope and timeline problems found in the first draft.

1. Goals and guiding decisions

The assignment is graded on Functionality, UI/UX, Database design, API design, Code quality, Modularity and Code understanding. Every decision below is made to score on those, with the synced transcript as the centerpiece.

Decision	Choice	Why
Frontend	Next.js 14 (App Router) + TypeScript, mostly client components	Every page depends on the API, so SSR adds little. A typed fetch layer is simplest.
Styling	Tailwind CSS + shadcn/ui (Radix primitives)	Modals, dropdowns, toasts and focus handling for free, so time goes into Fireflies fidelity. Vanilla CSS is possible but costs hours on accessibility.
Backend	FastAPI + SQLAlchemy 2.0 + Pydantic v2	Auto-generated /docs, clean layering, easy validation.
Database	SQLite with foreign keys enforced	Required by the brief. Enforcement must be switched on explicitly (see 3.3).
Transcription	Seeded and uploaded transcripts only	Explicitly allowed. Effort goes into UX.
AI features	Seeded summaries, plus optional LLM with a mock fallback	The app and demo must work with no API key.
Playback	Pluggable engine: real audio if available, otherwise a simulated clock	Seeded meetings are 20-45 minutes, so one short sample file would break seeking.

2. Architecture

```

Browser (Next.js)
  pages -> components -> hooks -> api client (typed fetch)
                                |
                                REST / JSON
                                |
FastAPI
  routers (HTTP only) -> services (business logic) -> models (SQLAlchemy) ->
SQLite
                                |
                                ai_service (LLM or mock)

```

Rules that keep the code modular:

- Routers only parse the request, call a service, and shape the response. No queries or business logic in routers.
- Services own transactions and rules (replace transcript, cascade behavior, parsing).
- Pydantic schemas are separate from ORM models. Never return ORM objects directly.
- Frontend components never call fetch directly. They use hooks, which use the api client.
- Core code ships first. Bonus routers, services and components are created only when you reach them, so no half-built files sit in the repo.

3. Database design

3.1 Entities

meetings	1 --- *	transcript_segments	
meetings	1 --- *	chapters	
meetings	1 --- 1	summaries	
meetings	1 --- *	action_items	
meetings	* --- *	participants	(via meeting_participants)
participants	1 --- *	transcript_segments	(speaker)
participants	1 --- *	action_items	(assignee)
meetings	* --- *	tags	(via meeting_tags) [bonus]
transcript_segments	1 --- *	comments	[bonus]

3.2 SQL schema

```

CREATE TABLE meetings (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  title       TEXT NOT NULL,
  date        DATETIME NOT NULL,
  duration_sec INTEGER NOT NULL DEFAULT 0,
  audio_url   TEXT,                                -- NULL => simulated
  playback
  source      TEXT NOT NULL DEFAULT 'manual',      -- 'seed' | 'manual' |
  'upload'
  created_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE participants (
  id    INTEGER PRIMARY KEY AUTOINCREMENT,
  name  TEXT NOT NULL,
  email TEXT UNIQUE                      -- nullable; unique when
  present
);

CREATE TABLE meeting_participants (
  meeting_id    INTEGER NOT NULL REFERENCES meetings(id) ON DELETE CASCADE,
  participant_id INTEGER NOT NULL REFERENCES participants(id) ON DELETE
  CASCADE,
  PRIMARY KEY (meeting_id, participant_id)
);

CREATE TABLE transcript_segments (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  meeting_id  INTEGER NOT NULL REFERENCES meetings(id) ON DELETE CASCADE,
  speaker_id  INTEGER REFERENCES participants(id) ON DELETE SET NULL,
  speaker_label TEXT NOT NULL,              -- raw label, e.g.
  'Speaker 2' from a VTT
  start_sec   REAL NOT NULL,
  end_sec     REAL NOT NULL,
  text        TEXT NOT NULL,
  position    INTEGER NOT NULL,            -- order within the
  meeting
  CHECK (end_sec >= start_sec)
);

CREATE TABLE summaries (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  meeting_id  INTEGER NOT NULL UNIQUE REFERENCES meetings(id) ON DELETE

```

```

CASCADE,
  overview      TEXT,
  keywords_json TEXT,                                -- JSON array of strings
(see note)
  generated_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE chapters (
  id            INTEGER PRIMARY KEY AUTOINCREMENT,
  meeting_id    INTEGER NOT NULL REFERENCES meetings(id) ON DELETE CASCADE,
  title         TEXT NOT NULL,
  description   TEXT,
  start_sec     REAL NOT NULL,
  position      INTEGER NOT NULL
);

CREATE TABLE action_items (
  id            INTEGER PRIMARY KEY AUTOINCREMENT,
  meeting_id    INTEGER NOT NULL REFERENCES meetings(id) ON DELETE CASCADE,
  assignee_id   INTEGER REFERENCES participants(id) ON DELETE SET NULL,
  text          TEXT NOT NULL,
  due_date      DATE,
  is_completed  BOOLEAN NOT NULL DEFAULT 0,
  created_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Bonus
CREATE TABLE tags (
  id    INTEGER PRIMARY KEY AUTOINCREMENT,
  name  TEXT NOT NULL UNIQUE
);

CREATE TABLE meeting_tags (
  meeting_id INTEGER NOT NULL REFERENCES meetings(id) ON DELETE CASCADE,
  tag_id     INTEGER NOT NULL REFERENCES tags(id) ON DELETE CASCADE,
  PRIMARY KEY (meeting_id, tag_id)
);

CREATE TABLE comments (
  id            INTEGER PRIMARY KEY AUTOINCREMENT,
  meeting_id    INTEGER NOT NULL REFERENCES meetings(id) ON DELETE CASCADE,
  segment_id    INTEGER NOT NULL REFERENCES transcript_segments(id) ON DELETE
CASCADE,
  body          TEXT NOT NULL,
  kind          TEXT NOT NULL DEFAULT 'comment',      -- 'comment' | 'highlight'
| 'soundbite'
  created_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

```

```
-- Indexes
CREATE INDEX idx_meetings_date          ON meetings(date DESC);
CREATE INDEX idx_segments_meeting_time ON transcript_segments(meeting_id,
start_sec);
CREATE INDEX idx_segments_meeting_pos   ON transcript_segments(meeting_id,
position);
CREATE INDEX idx_chapters_meeting       ON chapters(meeting_id, position);
CREATE INDEX idx_actions_meeting        ON action_items(meeting_id);
CREATE INDEX idx_mp_participant         ON
meeting_participants(participant_id);
CREATE INDEX idx_comments_meeting       ON comments(meeting_id);
```

3.3 Fixes from the first draft

Problem in v1	Fix
Participants belonged to one meeting, so the same person was duplicated and "filter by participant" was unreliable	Standalone <code>participants</code> table plus <code>meeting_participants</code> join table
Speaker and assignee were free-text strings	<code>speaker_id</code> / <code>assignee_id</code> foreign keys (SET NULL), with <code>speaker_label</code> kept for unknown speakers from uploads
Chapters stored as a JSON blob	Real <code>chapters</code> table, so chapters are queryable and seekable
<code>ON DELETE CASCADE</code> did nothing, because SQLite ignores foreign keys by default	Enable the pragma on every connection (below) and use ORM cascades
<code>comments.transcript_segment_id</code> had no delete rule	<code>ON DELETE CASCADE</code>
<code>status</code> column with states nothing produced	Removed. Replaced with <code>source</code>
<code>avatar_color</code> stored in the DB	Derived on the frontend by hashing the name
No media reference	<code>audio_url</code> on meetings
<code>due_date</code> as DATETIME	<code>DATE</code>

One deliberate trade-off to be ready to defend: `keywords_json` stays a JSON text column because keywords are display-only. If you want to filter by topic, use the `tags` table instead.

Enforce foreign keys on every connection:

```
from sqlalchemy import event
from sqlalchemy.engine import Engine

@event.listens_for(Engine, "connect")
def set_sqlite_pragma(dbapi_conn, _):
    cur = dbapi_conn.cursor()
    cur.execute("PRAGMA foreign_keys=ON")
    cur.close()
```

Also set `cascade="all, delete-orphan"` on the Meeting relationships so the ORM and the database agree.

4. API contract

Base path `/api`. JSON everywhere except the upload endpoint (multipart) and export (file download). Use `PATCH` for partial updates, `204` for deletes, `404` for missing resources and `422` for validation errors.

4.1 Error format (consistent, so the frontend can show toasts)

```
{ "error": { "code": "validation_error", "message": "Title is required",
"details": [] } }
```

4.2 Core endpoints

Method	Path	Notes
GET	<code>/api/meetings</code>	Query: <code>q</code> , <code>participant_id</code> , <code>tag</code> , <code>date_from</code> , <code>date_to</code> , <code>sort</code> (<code>date_desc</code> default, <code>date_asc</code> , <code>title</code>), <code>page</code> , <code>page_size</code> . Returns <code>{items, total, page, page_size}</code>
POST	<code>/api/meetings</code>	JSON: <code>title</code> , <code>date</code> , <code>participants</code> (existing ids or new <code>{name, email}</code>), optional <code>transcript_text</code> for pasted transcripts. Returns <code>201</code>

Method	Path	Notes
POST	/api/meetings/upload	Multipart: file (.txt / .vtt / .json), optional title and date. Parses and creates the meeting. Returns 201
GET	/api/meetings/{id}	Metadata, participants, tags
PATCH	/api/meetings/{id}	Title, date, participant ids, tag ids
DELETE	/api/meetings/{id}	204 , cascades to all child rows
GET	/api/meetings/{id}/transcript	All segments ordered by position
PUT	/api/meetings/{id}/transcript	Replace the whole transcript in one transaction (see 4.4)
GET	/api/meetings/{id}/summary	Overview, keywords, chapters
POST	/api/meetings/{id}/summary/regenerate	LLM if a key exists, otherwise mock generator
GET	/api/meetings/{id}/action-items	List
POST	/api/meetings/{id}/action-items	Create
PATCH	/api/action-items/{id}	Edit text, assignee, due date or is_completed . No separate toggle endpoint
DELETE	/api/action-items/{id}	204
GET	/api/participants	For filters and pickers

Action items are updated through a flat route, so the client does not need the meeting id to edit one. Naming is `action-items` everywhere.

4.3 Bonus endpoints (build only after the core is deployed)

Method	Path	Notes
GET	/api/search?q=&type=	Global search across meeting titles, transcripts, summaries
GET / POST	/api/tags	List and create
PUT	/api/meetings/{id}/tags	Set the tag list for a meeting

Method	Path	Notes
GET / POST	<code>/api/meetings/{id}/comments</code>	Comments, highlights, soundbites
DELETE	<code>/api/comments/{id}</code>	204
GET	<code>/api/meetings/{id}/export? format=md txt</code>	Markdown and TXT. PDF only if time remains (see 8)
POST	<code>/api/meetings/{id}/chat</code>	Ask a question. Mock answers if no API key

4.4 Defined behaviors

- **Replacing a transcript** deletes old segments and inserts new ones in a single transaction. Comments on old segments cascade away. The UI shows a confirmation modal first.
- **Upload parsing:** VTT and JSON are supported properly. Plain `.txt` accepts lines like `Speaker: text` with optional `[mm:ss]` prefixes. Any file that cannot be parsed returns 422 with a message that appears in a toast. Maximum file size 2 MB.
- **Speaker matching:** on create or upload, speaker labels are matched to participants by name (case-insensitive). Unmatched labels keep `speaker_id = NULL`.
- **Library search (q)** is server-side, matching title and participant name. **In-transcript search** is client-side, because the full transcript is already loaded and it needs instant highlighting. Know why you split it this way.

5. Frontend plan

5.1 Routes

<code>/</code>	Meetings library
<code>/meetings/[id]</code>	Meeting detail
<code>/search</code>	Global search (bonus)
<code>/settings</code>	Placeholder
<code>/integrations</code>	Coming soon placeholder

5.2 Structure

```

frontend/src/
  app/                layout.tsx, page.tsx, meetings/[id]/page.tsx,
settings/, search/
  components/
    layout/           Sidebar, Navbar, AppShell
    meetings/         MeetingCard, MeetingList, MeetingFilters,
MeetingFormModal, DeleteDialog
    transcript/       TranscriptPanel, TranscriptLine, TranscriptSearch
    player/           MediaPlayer, SeekBar
    summary/          SummaryPanel, ChaptersList, ActionItemList,
ActionItemRow
    ui/               shadcn components (Button, Dialog, Toast, Tabs, ...)
    hooks/            useMeetings, useMeeting, useTranscript, usePlayer,
useDebounce
    lib/              api.ts (typed client), playback/ (engines), format.ts,
avatarColor.ts
    types/            api.ts (mirrors backend schemas)

```

5.3 The synced player (most important feature)

Playback is behind one small interface, so the transcript never cares what is playing:

```

interface PlaybackEngine {
  play(): void;
  pause(): void;
  seek(sec: number): void;
  getTime(): number;
  subscribe(cb: (t: number, playing: boolean) => void): () => void;
}

```

- `AudioEngine` wraps an HTML5 `<audio>` element and is used when `audio_url` exists.
- `SimulatedEngine` runs a clock with `requestAnimationFrame`. It is the default for seeded meetings, so durations always match the transcript.
- `usePlayer(meeting)` exposes `currentTime`, `isPlaying`, `seek`, and `activeSegmentIndex`.
- Finding the active segment uses **binary search** over `start_sec`, not a linear scan.
- Sync is bidirectional: clicking a line calls `seek(segment.start_sec)`, and playback updates the highlighted line. The active line auto-scrolls into view, but auto-scroll pauses when the user scrolls manually (show a "Jump to current" button).
- `TranscriptLine` is memoized, so only the old and new active lines re-render on each tick.

- Clicking a chapter also seeks. Keyboard shortcuts: space to play and pause, arrows to skip 5 seconds.

5.4 Library page

- Search input with a 300 ms debounce, participant filter, date range filter, sort control, and pagination or infinite scroll.
- URL query params mirror the filters, so a refresh or shared link keeps them.
- Skeleton loaders, empty state with a call to action, and error state with retry.
- Create meeting modal with three tabs: Form, Paste transcript, Upload file. Edit modal for title, date and participants. Delete confirmation dialog.

5.5 Detail page

- Header: title, date, duration, participants, edit and delete menu, export (bonus).
- Left: player with seek bar above the transcript, with in-transcript search, match count, and next/previous match.
- Right: tabs for Summary, Action items, Chapters. Action item completion uses **optimistic updates with rollback and an error toast** if the request fails.
- Fireflies-style details: speaker avatars, per-segment timestamps, a sticky player bar.

5.6 Placeholders ("Coming soon" is enough)

Live bot, real transcription, integrations (Zoom, Meet, calendar, CRM), team and sharing, authentication (assume a default logged-in user), and settings sections. Make them look intentional: real sidebar entries leading to a clean coming-soon page.

6. Design system

The hex values below are working approximations. **Before building, open the real Fireflies app and sample the actual colors, spacing and sidebar styling from screenshots**, then put them in `tailwind.config.ts` as theme tokens.

Token	Light	Dark
Primary	#7A5AF8	#7A5AF8
Primary hover	#6941C6	#8B6FF9
Primary subtle	#F4F0FF	#252244
Background	#FFFFFF	#0D0B21

Token	Light	Dark
Surface	#F9FAFB	#1A1736
Border	#E5E7EB	#2E2A52
Text primary	#111827	#FFFFFF
Text secondary	#6B7280	#9CA3AF
Success / Warning / Error	#10B981 / #F59E0B / #EF4444	same

Typography: Inter, 24-32 px headings (600-700), 14-16 px body, 12-13 px captions. Cards use 12 px radius with a subtle border and hover shadow. Buttons 40 px high. Modals use a blurred backdrop. Toasts appear bottom-right and auto-dismiss. Sidebar about 240 px wide with icon and label navigation.

7. Seed data

Seed data is what makes the demo look finished, and it takes real time to write.

- Six meetings (titles as in the first draft: Q4 Product Roadmap Planning, Engineering Sprint Retrospective, Client Onboarding Acme Corp, Weekly Marketing Sync, Design Review Dashboard Redesign, Investor Update Call).
- Around 8-10 shared participants, so participant filtering is actually interesting. Several people appear in multiple meetings.
- Each meeting has 40-80 timestamped segments, a summary, keywords, 3-6 chapters whose `start_sec` matches real segment times, 3-5 action items (some completed, some with due dates), and tags.
- **Generate it once with an LLM, review it, and commit it as JSON** in `backend/app/seed/data/`. A loader script reads that JSON. This avoids hand-writing hundreds of lines of dialogue.
- The seed loader is **idempotent**: it only runs when the meetings table is empty, and it runs automatically at startup. This also covers deployments where the disk resets.

8. Realistic timeline (24 hours, with slack)

The first draft added up to exactly 24 hours with every bonus included and no buffer. This version finishes and deploys the core first.

#	Phase	Hours
0	Setup, repo structure, decide tokens from real Fireflies screenshots	1
1	Backend core: models, pragma, schemas, meetings CRUD, participants, action items, parsers	4
2	Seed data: generate, review, loader	2
3	Frontend foundation and library page (layout, api client, filters, create/edit/delete, toasts)	4
4	Detail page: engines, synced transcript, search, summary, chapters, action items	5
5	First deployment checkpoint (both services live, seeded, CORS verified)	1
6	Polish, empty/error/loading states, backend tests, README	2.5
7	Buffer for the unexpected	2
8	Bonuses, in order (below)	2.5
	Total	24

Bonus order, cheapest and most visible first: dark mode, Markdown/TXT export, tags and tag filter, global search, comments and highlights, mocked or real chat. PDF export last or skipped, since PDF libraries often fail on free hosting. If you want PDF, generate it client-side from the Markdown export.

Cut order if time runs short: chat, comments, PDF, global search, tags. Never cut: synced transcript, CRUD with persistence, seed data, README, deployment.

9. Deployment

- **Frontend:** Vercel, with `NEXT_PUBLIC_API_URL` set to the backend URL.
- **Backend:** Render or Railway. Configure CORS for the Vercel domain and localhost.
- **Persistence:** free-tier disks are ephemeral and SQLite resets on restart or redeploy. Either attach a persistent volume, or rely on the idempotent startup seed and state the limitation clearly in the README (user-created data may reset).
- Deploy a minimal version at the end of Phase 3, not on the last day, so hosting problems surface early.

- Add a `/api/health` endpoint. Remember that free instances sleep, so the first request can be slow: mention it in the README.
- Environment variables: `DATABASE_URL` , `CORS_ORIGINS` , optional `LLM_API_KEY` , `NEXT_PUBLIC_API_URL` . Commit a `.env.example` for both services.

10. Testing (small but visible)

Backend (pytest + FastAPI TestClient, in-memory SQLite):

- Meetings CRUD, including 404 and 422 cases.
- Deleting a meeting removes segments, chapters, action items and join rows (proves cascades and the pragma work).
- VTT, JSON and TXT parsers, including malformed input.
- Filtering by participant, date range and search query.
- Replace-transcript transaction.

Frontend (optional): unit tests for the binary-search active-segment function and the transcript search highlighter.

11. README checklist

- Overview and screenshots.
- Tech stack and rationale.
- Setup for backend and frontend (exact commands), environment variables, how seeding works.
- Architecture description and diagram.
- Database schema with an ER diagram and the reasoning behind key choices.
- API overview table, plus a pointer to `/docs` .
- Assumptions: default logged-in user, mocked transcription and AI, simulated playback.
- Known limitations: ephemeral SQLite on free hosting, cold starts, mocked features.
- Deployed link and repository link.

12. Definition of done

- ☐ Library: list, search, participant and date filters, sort, create, edit, delete
- ☐ Detail: transcript with speakers and timestamps, working player with seek bar
- ☐ Click a line to seek, and playback highlights the current line
- ☐ In-transcript search with highlights and next/previous

- ☐ Summary, chapters (click to seek), and action items with add, edit, complete and delete, all persisted
- ☐ Toasts, modals, loading, empty and error states
- ☐ Settings and integrations placeholders
- ☐ Six seeded meetings, loaded automatically
- ☐ Deployed frontend and backend, public GitHub repo, complete README
- ☐ You can explain every file

13. Interview preparation

Be ready to explain:

1. Why participants are many-to-many, and why segments are rows instead of a JSON blob.
2. Why SQLite needs `PRAGMA foreign_keys=ON`, and how cascades work at both the ORM and database levels.
3. How the active transcript line is found (binary search), why `TranscriptLine` is memoized, and how auto-scroll avoids fighting the user.
4. Why the player uses an engine interface and a simulated fallback.
5. How optimistic updates roll back on failure.
6. Why library search is server-side and transcript search is client-side.
7. How uploads are parsed and how bad files are handled.
8. Your deployment trade-offs, especially SQLite persistence.